Lab 4: Simple Python Database Connections

Solution document

Student identity: faisal-yusra@ubuntu

Databases fork: https://github.com/faisal-yusra/Databases

Overview

I edited one Python file, `db_connect.py`, through four stages: MySQL with values in the code, MongoDB with values in the code, MongoDB with values from `os.getenv()`, and MongoDB with values from a credentials file. Python ran directly on the Ubuntu server. Docker ran only the databases.

Task 1: Fork and clone the Databases repository

I forked `WaqasSaleem97/Databases` to my account, keeping the name `Databases`, and cloned my own fork over SSH. The fork holds the code I edit; my separate DevOps repository holds the evidence.

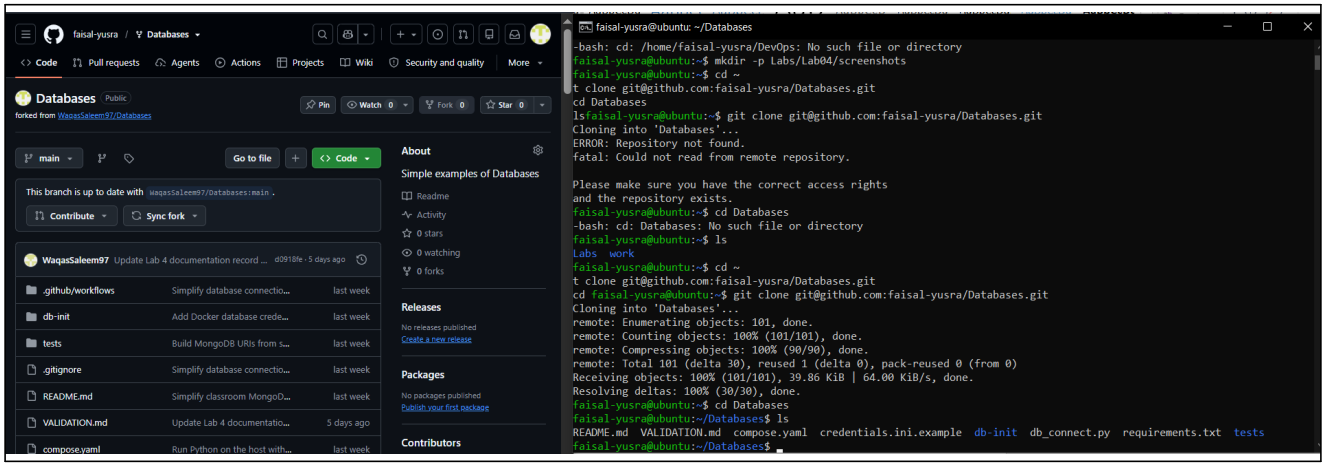


Figure: My fork faisal-yusra/Databases (forked from WaqasSaleem97/Databases) beside the Ubuntu terminal (*databases_fork.png*)

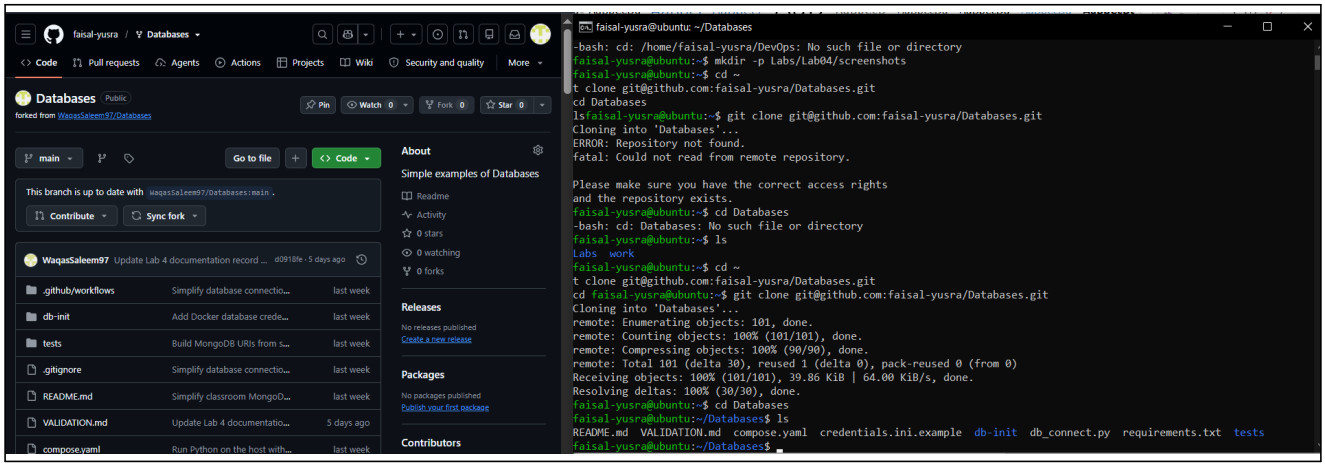


Figure: Cloning my fork and listing its files (*databases_clone.png*)

Task 2: Install Docker, Docker Compose and Python

I installed `docker.io` and `curl`, added the Docker Compose plug-in manually, started Docker and added my user to the `docker` group. After signing out and back in, Git (2.53.0), Docker (29.1.3) and Docker Compose (v5.6.1) all report their versions. I then created a Python virtual environment in ~/Databases and

installed pymongo, mysql-connector-python and dnspython.

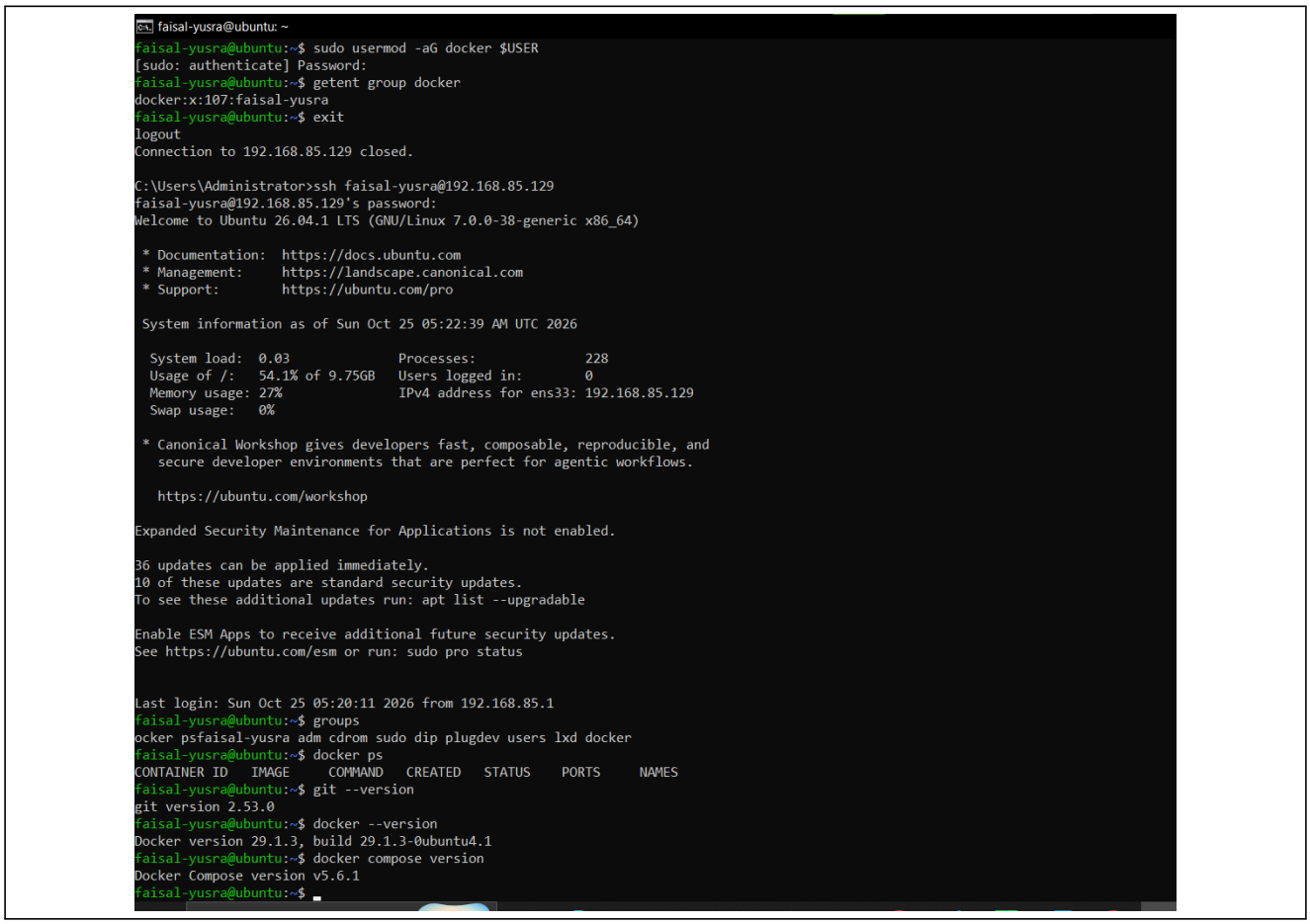


Figure: git, docker and docker compose versions on the Ubuntu server (*docker_installation.png*)

Task 3: Start the database services

`docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod` started four services, all healthy. On the Ubuntu host they use ports 3307 (MySQL), 27017 (MongoDB dev), 27018 (MongoDB test) and 27019 (MongoDB simulated prod).

Commands: `docker compose ps`, then `docker compose port mysql 3306`, `docker compose port mongo-dev 27017`, `docker compose port mongo-test 27017` and `docker compose port mongo-prod 27017`.



Figure: All four database services reported as healthy (*database_services.png*)

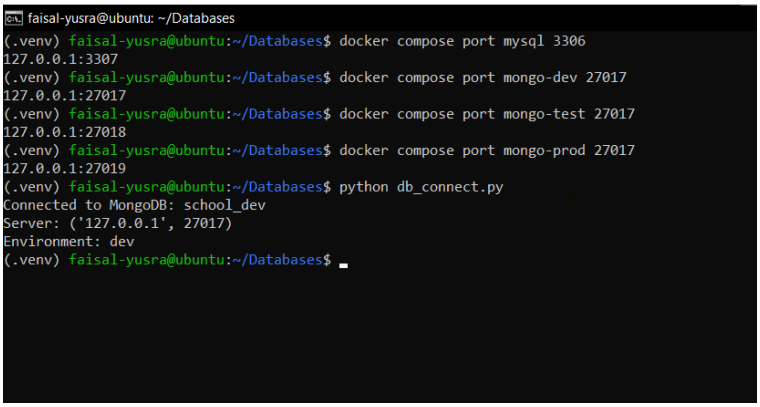


Figure: Host ports of the four database services (*database_ports.png*)

Task 4: MySQL with values in the code

Result: *Connected to MySQL: school_mysql*, *Server: 127.0.0.1 3307*, *Environment: dev*. `user` and `password` supply the credentials, `host` and `port` identify the MySQL server, `database` selects the database, and `mysql.connector.connect()` opens the connection. The query reads the environment name stored in MySQL, and closing the cursor and connection releases the resources.

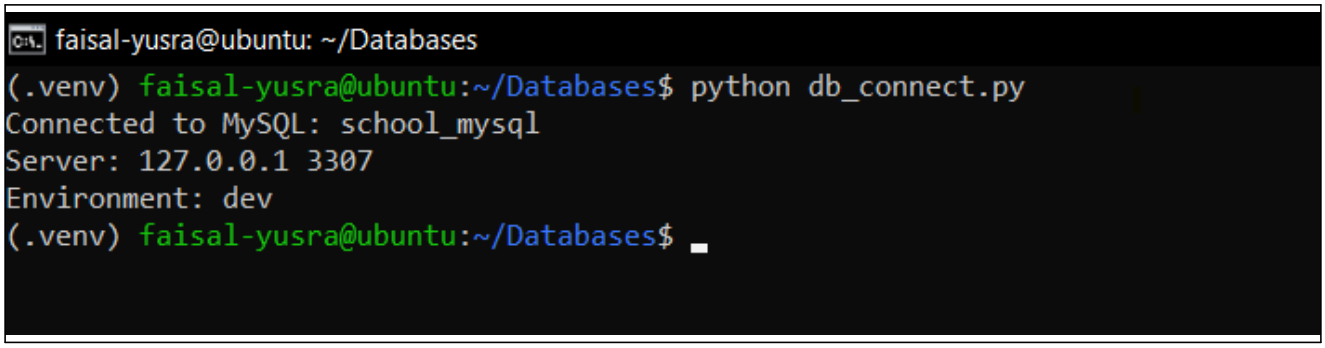


Figure: MySQL connection with the settings written in the code (*mysql_hardcoded_connection.png*)

Task 5: MongoDB with values in the code

Result: *Connected to MongoDB: school_dev*, server 127.0.0.1 port 27017, *Environment: dev*. The username, password, endpoint and database name are held in separate variables and joined into a `mongodb://` URI. `database.command("ping")` sends a real request, because creating a client or selecting a database alone does not prove that the server is reachable.

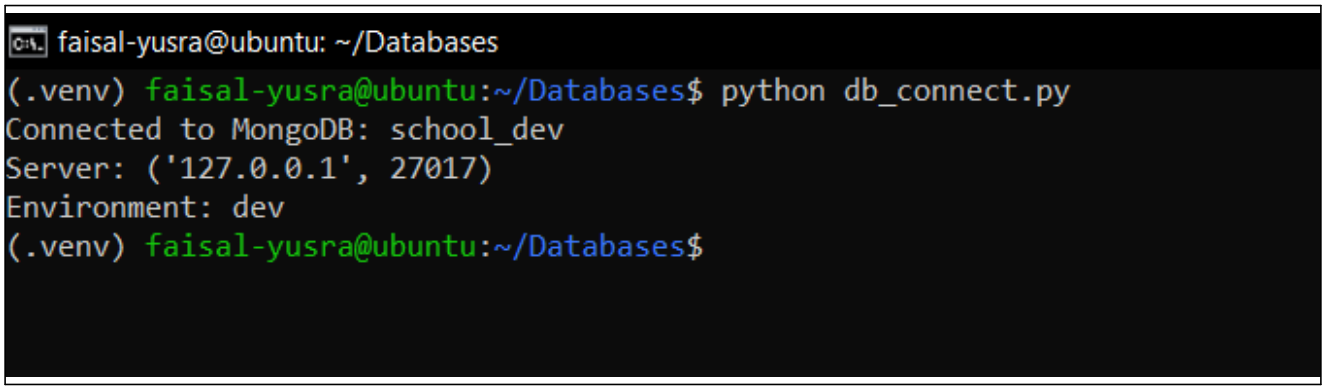


Figure: MongoDB dev connection with the settings written in the code (*mongo_hardcoded_connection.png*)

Task 6: MongoDB with os.getenv()

The same program connected to different MongoDB deployments by changing only four exported variables (`MONGO_USERNAME`, `MONGO_PASSWORD`, `MONGO_ENDPOINT`, `MONGO_DATABASE`). Python inherits these variables from the shell and joins them into the connection URI. Each run printed its own database name, port and stored environment value.

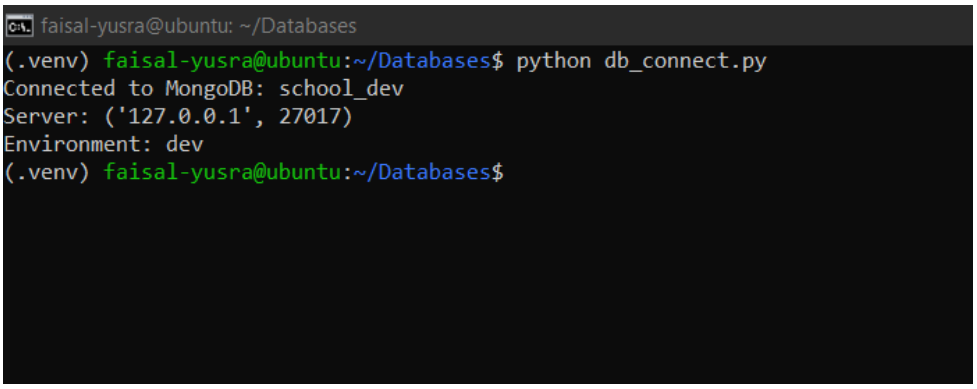


Figure: Dev connection using environment variables (school_dev, port 27017) (*mongo_env_dev.png*)



Figure: Test connection using environment variables (school_test, port 27018) (*mongo_env_test.png*)

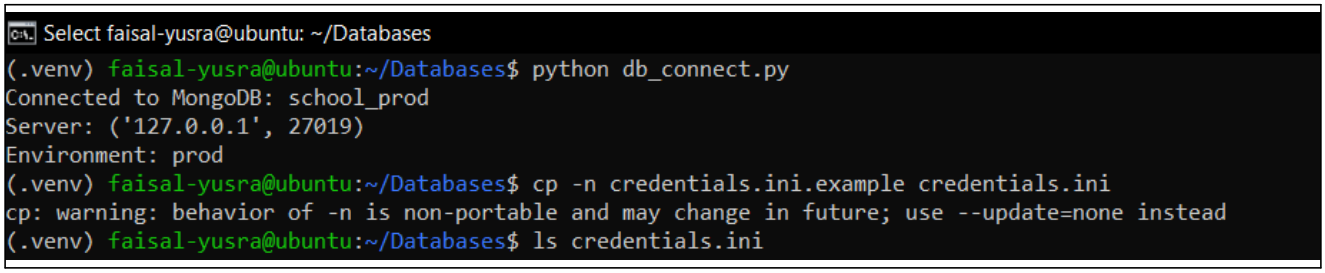


Figure: Simulated prod connection using environment variables (school_prod, port 27019) (*mongo_env_prod.png*)

Task 7: MongoDB with a credentials file

I created `credentials.ini` from the supplied example and restricted it with `chmod 600`. It has `[dev]`, `[test]` and `[prod]` sections. `configparser` reads the section named by APP_ENV (default dev), and the MongoDB connection block is unchanged. An INI file stores plaintext, so it stays local and is ignored by Git.

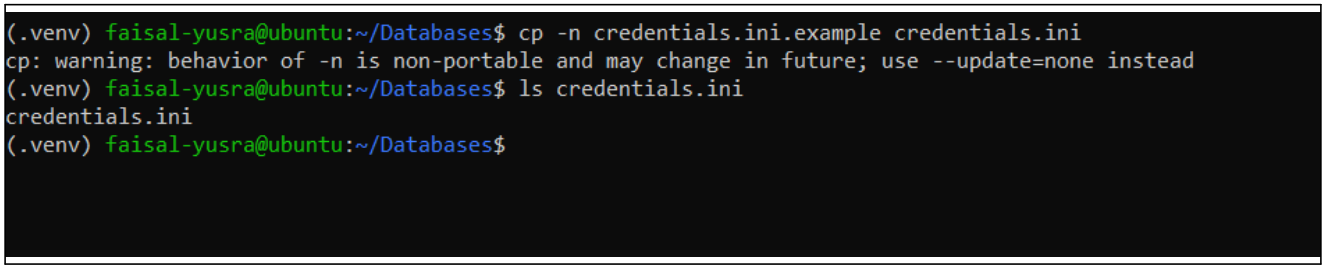


Figure: credentials.ini created from the example file (*credential_files_created.png*)

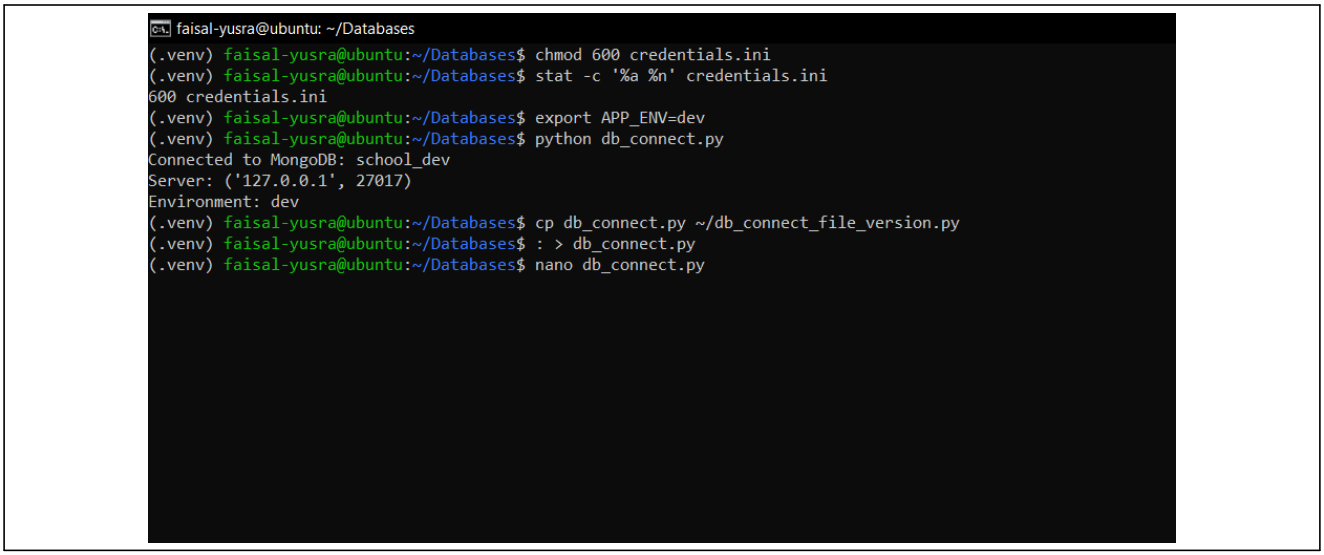


Figure: File mode 600 on credentials.ini, followed by the dev connection (APP_ENV=dev) (*credential_file_permissions.png*)

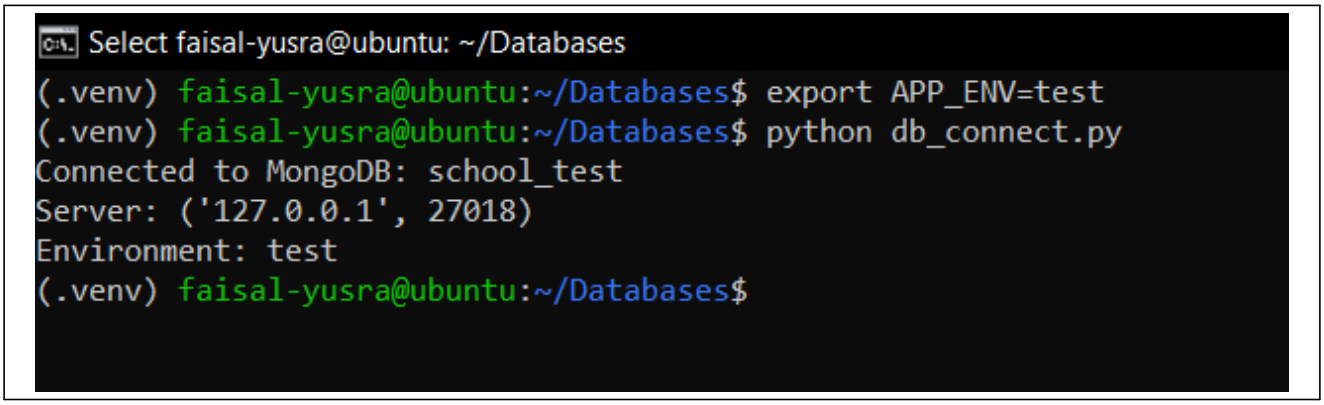


Figure: Test connection using the credentials file (APP_ENV=test) (*mongo_file_test.png*)

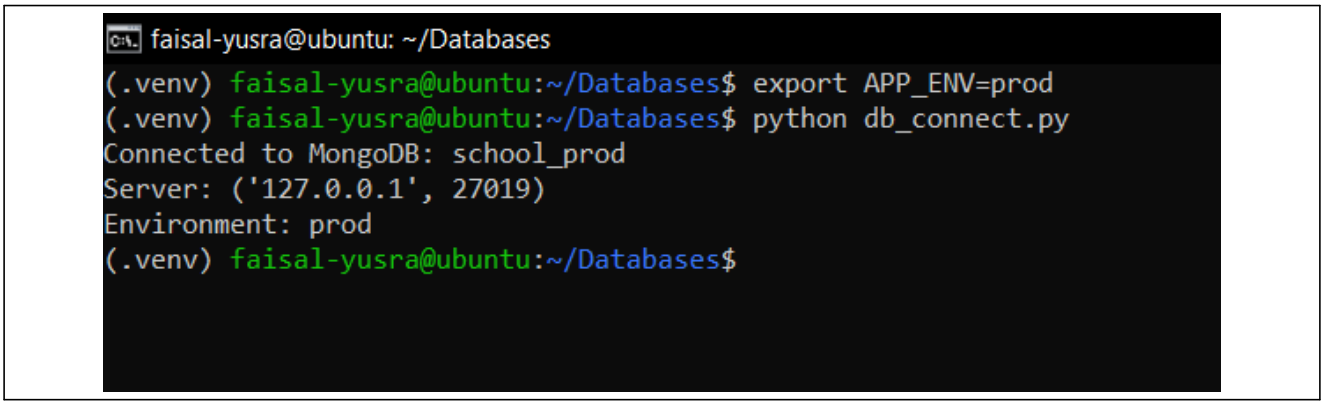


Figure: Simulated prod connection using the credentials file (APP_ENV=prod) (*mongo_file_prod.png*)

The dev run from the credentials file is also shown in the permissions figure above (*mongo_file_dev.png* is the same capture).

Problems encountered and fixes

MongoDB would not start. The container exited with a message that Linux kernel 6.19 and newer is incompatible with this MongoDB version. My Ubuntu 26.04 server runs kernel 7.0. I set `GLIBC_TUNABLES=glibc.pthread.rseq=1` for the three MongoDB services in a local `compose.override.yaml`, which is a published workaround.

Memory and disk. The VM became very slow and the root disk filled up. I enlarged the root logical volume with `lvextend` and started the services one at a time.

Authentication failed in the health check. The earlier interrupted starts had left a database volume without its users. `docker compose down -v` reset the disposable volumes so the initialization scripts ran again.

Conclusion

The same connection pattern worked for MySQL and MongoDB. Moving the settings from the code into environment variables and then into a credentials file lets one program target dev, test and simulated prod without editing its logic. Each target was verified using the database name, the endpoint and the environment value stored in the database.